

RNNs, LSTMs, and Transformers for Time Series

A time-series view

Mikołaj Słupiński

May 2026

After this lecture you should be able to:

- Turn a time series into supervised learning examples.
- Explain why recurrence is a natural model for ordered measurements.
- Diagnose vanishing and exploding gradients in recurrent training.
- Describe how LSTM and GRU gates control memory.
- Compare RNNs, modern state-space models, and Transformers for forecasting.

1. Time-series problem setup
2. RNNs and backpropagation through time
3. LSTM and GRU cells
4. Modern recurrent and state-space models
5. Transformers for time series
6. What comes after Transformers?

Time-series problem setup

What is a time series?

A time series is an ordered sequence of observations

$$\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_T, \quad \mathbf{x}_t \in \mathbb{R}^C$$

where C is the number of channels or variables.

Examples:

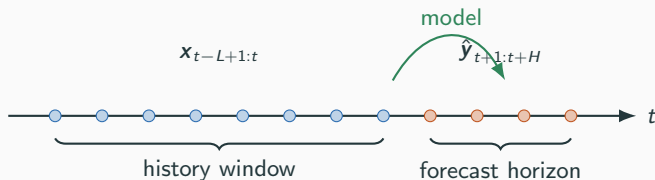
- electricity load every 15 minutes,
- daily retail demand for many products,
- ECG, accelerometer, or industrial sensor streams,
- prices, returns, volume, and other financial measurements.

Common time-series tasks

- **Forecasting:** predict future values or quantiles.
- **Classification:** assign a label to a window, e.g. activity or fault type.
- **Anomaly detection:** flag behavior unlikely under normal dynamics.
- **Imputation:** fill missing measurements.
- **Filtering:** estimate a latent state from noisy observations.

All of these require modeling how information moves through time.

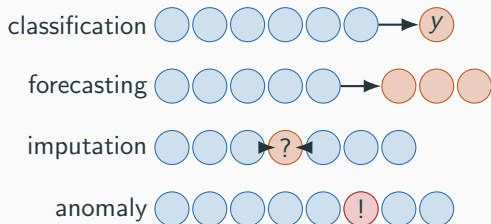
Forecasting as supervised learning



$$f_{\theta} : \mathbf{x}_{t-L+1:t} \mapsto \hat{\mathbf{y}}_{t+1:t+H}$$

Key choices: history length L , forecast horizon H , target type, and evaluation metric.

Four sequence prediction shapes



The same recurrent machinery can be used for all four, but the loss is attached at different time steps.

Multivariate time series

A multivariate sequence has two axes:

$$\mathbf{X} = \begin{bmatrix} x_{1,1} & x_{1,2} & \cdots & x_{1,C} \\ x_{2,1} & x_{2,2} & \cdots & x_{2,C} \\ \vdots & \vdots & & \vdots \\ x_{T,1} & x_{T,2} & \cdots & x_{T,C} \end{bmatrix}$$

- **Temporal dependencies:** what happened before?
- **Cross-channel dependencies:** which variables move together?
- **Exogenous variables:** calendar, weather, prices, events.

Point forecasts are often not enough.

Instead of only

$$\hat{y}_{t+1} = f_{\theta}(\mathbf{x}_{1:t}),$$

we may model

$$p_{\theta}(y_{t+1:t+H} \mid \mathbf{x}_{1:t}).$$

This supports prediction intervals, quantiles, scenario generation, and risk-aware decisions.

Evaluation metrics

Task	Typical metric	What it emphasizes
Point forecast	MAE, RMSE	average error
Probabilistic forecast	NLL, CRPS	calibrated uncertainty
Classification	accuracy, F1	labels per window
Anomaly detection	AUROC, precision at k	rare events

The model that wins under RMSE may be a poor decision model if uncertainty matters.

Time-series pitfalls

- Random train-test splits can leak future information.
- Scaling statistics must be fit on the training period only.
- Calendar features may encode deployment assumptions.
- Backtests should respect decision time: what was known when?
- Long horizons often need different evaluation than one-step forecasts.

Sequence models do not fix leakage; they can exploit it very effectively.

Recurrent neural networks

Why recurrence?

For a forecasting model, the input length may vary:

$$\mathbf{x}_{1:t} = \mathbf{x}_1, \dots, \mathbf{x}_t.$$

A recurrent model keeps a state

$$\mathbf{h}_t = F_{\theta}(\mathbf{h}_{t-1}, \mathbf{x}_t)$$

that summarizes the past for future predictions.

This is the neural-network version of a filter: update a belief as new measurements arrive.

Hidden state as compressed history

$$\mathbf{h}_t \approx \text{useful summary of } \mathbf{x}_{1:t}.$$

- Good state: remembers seasonality, level shifts, recent shocks, and regimes.
- Bad state: forgets useful old events or overreacts to noise.
- The same update rule is reused at every time step.

The key design question is what kind of memory the state can represent.

Vanilla RNN equations

A simple RNN layer is:

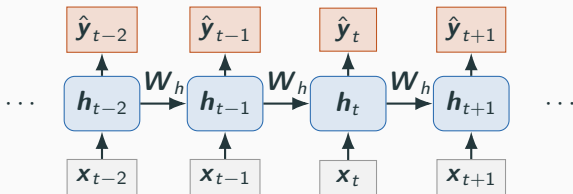
$$\mathbf{h}_t = \tanh(\mathbf{W}_x \mathbf{x}_t + \mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{b}),$$

$$\hat{\mathbf{y}}_t = g_\theta(\mathbf{h}_t).$$

Same parameters, many time steps:

- $\mathbf{W}_x$ maps new observations into state space,
- $\mathbf{W}_h$ transports memory forward,
- g_θ reads predictions from the state.

Unrolling an RNN



After unrolling, an RNN becomes a deep network whose depth is the sequence length.

RNN as a dynamical system

Ignore inputs for a moment:

$$\mathbf{h}_t = \tanh(\mathbf{W}_h \mathbf{h}_{t-1}).$$

The hidden state evolves like a nonlinear dynamical system.

- Eigenvalues of $\mathbf{W}_h$ influence stability.
- Repeated multiplication can shrink or amplify signals.
- The same issue appears when gradients flow backward.

Based on the dynamical-system explanation in 10-11-RNNs-from_TMLSS.pdf.

Backpropagation through time

For a loss over the whole window,

$$\mathcal{L} = \sum_{t=1}^T \ell(\hat{\mathbf{y}}_t, \mathbf{y}_t),$$

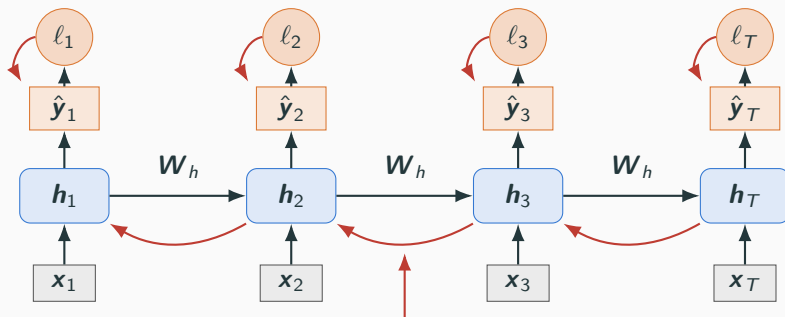
we differentiate through the unrolled graph.

This is BPTT:

- forward pass stores states $\mathbf{h}_1, \dots, \mathbf{h}_T$,
- backward pass sends error through time,
- parameter gradients are summed across repeated uses.

BPTT as error flow

losses send gradient contributions
to every reused copy of the cell



backward error flow through stored states

- each ℓ_t contributes a local error signal,
- errors move backward through the hidden-state links,
- shared parameters such as W_h receive summed gradients.

Many-to-one classification

For activity recognition or fault detection:

$$\hat{y} = g_{\theta}(\mathbf{h}_T).$$

- The final state must summarize the whole window.
- Supervision appears only at the end.
- Long windows make gradient flow especially important.

This is often harder than forecasting with a loss at every time step.

Sequence forecasting with an RNN

For rolling one-step prediction:

$$\mathbf{h}_t = F_{\theta}(\mathbf{h}_{t-1}, \mathbf{x}_t), \quad \hat{\mathbf{y}}_{t+1} = g_{\theta}(\mathbf{h}_t).$$

For multi-step forecasts:

- direct: predict all H steps at once,
- recursive: feed predictions back as future inputs,
- sequence-to-sequence: encode history, decode horizon.

Teacher forcing for time series

During training a decoder may receive the true previous value:

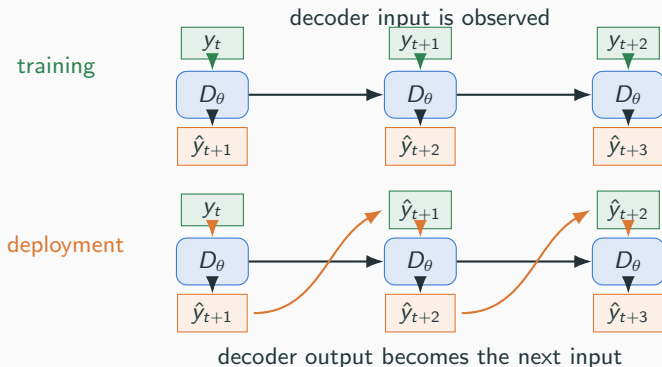
$$\hat{y}_{t+1} = g_{\theta}(\mathbf{h}_t, y_t).$$

During deployment it receives its own previous prediction.

- This mismatch can cause error accumulation.
- Scheduled sampling mixes true and generated values during training.
- Direct multi-horizon prediction avoids some feedback errors.

Teacher forcing and scheduled sampling are discussed in the RNN source deck.

Teacher forcing: what changes at inference?



- Training breaks the feedback loop by feeding observed y values into each decoder step.
- Forecasting closes the loop: $\hat{y}_{t+k}$ becomes part of the input for predicting $\hat{y}_{t+k+1}$.
- A small one-step error can move later inputs away from the training distribution.

Reducing teacher-forcing mismatch

Scheduled sampling replaces the decoder input by a mixture:

$$\tilde{y}_t = \begin{cases} y_t, & \text{with probability } p_t, \\ \hat{y}_t, & \text{with probability } 1 - p_t. \end{cases}$$

- Large p_t : stable early training, close to teacher forcing.
- Smaller p_t : the model sees its own imperfect predictions.
- The schedule should be judged on rolled-out horizon error, not only one-step loss.

Alternative: direct multi-horizon forecasting

$$f_{\theta}(\mathbf{x}_{t-L+1:t}) = \hat{\mathbf{y}}_{t+1:t+H}.$$

This removes recursive feedback, but asks one model output to cover the full horizon.

The temporal Jacobian contains a product:

$$\frac{\partial \mathbf{h}_T}{\partial \mathbf{h}_t} = \prod_{k=t+1}^T \frac{\partial \mathbf{h}_k}{\partial \mathbf{h}_{k-1}}.$$

If typical singular values are below 1, gradients decay exponentially.

Consequence: the model struggles to learn dependencies across many time steps.

Exploding gradients

If typical singular values are above 1, the same product can explode.

Symptoms:

- unstable loss curves,
- huge parameter updates,
- NaNs or saturated hidden states.

Exploding gradients are mostly an optimization stability problem.

Gradient clipping

Global norm clipping rescales large gradients:

$$\mathbf{g} \leftarrow \mathbf{g} \cdot \min \left(1, \frac{\tau}{\|\mathbf{g}\|_2} \right).$$

Practical rule:

- monitor gradient norms,
- choose a threshold that clips only some updates,
- combine with sensible initialization and learning rates.

Long short-term memory

Why LSTM?

The LSTM was designed to make long-range credit assignment easier.

Core idea:

- separate memory cell c_t from exposed hidden state h_t ,
- use gates to decide what to forget, write, and reveal,
- preserve useful information with near-identity paths.

Hochreiter and Schmidhuber, 1997; RNN source deck.

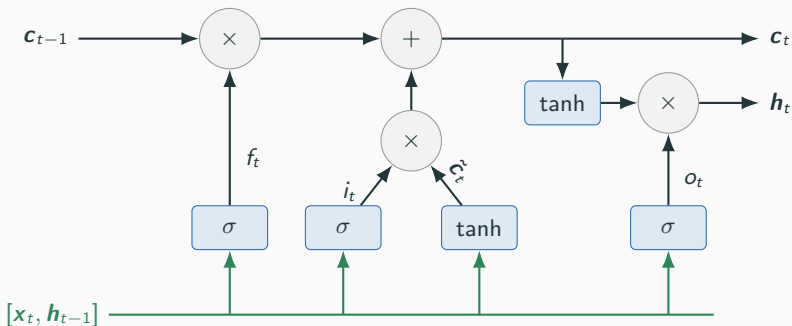
A plain recurrence repeatedly transforms the entire state.

An LSTM has a cell state:

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t.$$

The additive update gives memory a more direct path through time.

LSTM gates



Gates decouple **whether** to store information from **what** information to store.

$$\begin{aligned}f_t &= \sigma(\mathbf{W}_f[\mathbf{x}_t, \mathbf{h}_{t-1}] + \mathbf{b}_f), \\i_t &= \sigma(\mathbf{W}_i[\mathbf{x}_t, \mathbf{h}_{t-1}] + \mathbf{b}_i), \\\tilde{\mathbf{c}}_t &= \tanh(\mathbf{W}_c[\mathbf{x}_t, \mathbf{h}_{t-1}] + \mathbf{b}_c), \\\mathbf{c}_t &= f_t \odot \mathbf{c}_{t-1} + i_t \odot \tilde{\mathbf{c}}_t, \\o_t &= \sigma(\mathbf{W}_o[\mathbf{x}_t, \mathbf{h}_{t-1}] + \mathbf{b}_o), \\\mathbf{h}_t &= o_t \odot \tanh(\mathbf{c}_t).\end{aligned}$$

Each gate is a vector of values between 0 and 1.

$$f_t = \sigma(\mathbf{W}_f[\mathbf{x}_t, \mathbf{h}_{t-1}] + \mathbf{b}_f).$$

Time-series interpretation:

- keep slow seasonal state,
- drop a temporary shock after it stops mattering,
- reset memory after a regime change.

The forget gate is often initialized with positive bias so memory is not erased too early.

$$i_t \odot \tilde{\mathbf{c}}_t$$

controls what new information enters memory.

Example: after a sudden demand spike, the model must decide whether it is

- noise,
- a one-off promotion,
- a persistent level shift.

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t).$$

The cell can store information that is not always exposed.

Example: a yearly seasonal phase may be useful internally, even if the next-hour forecast mostly depends on recent load.

How LSTM helps gradients

The derivative through the cell includes:

$$\frac{\partial \mathbf{c}_t}{\partial \mathbf{c}_{t-1}} \approx f_t.$$

When f_t is close to 1, gradients can travel further back.

The LSTM does not magically solve all long-term learning, but it gives the optimizer a much better path.

A common LSTM forecaster:

$$\mathbf{h}_t, \mathbf{c}_t = \text{LSTM}(\mathbf{x}_t, \mathbf{h}_{t-1}, \mathbf{c}_{t-1}),$$

$$\hat{\mathbf{y}}_{t+1:t+H} = \text{Head}(\mathbf{h}_t).$$

The head may output a point forecast, quantiles, or distribution parameters.

- Start with one or two recurrent layers.
- Add a linear or MLP head for the horizon.
- Normalize inputs per channel using training-only statistics.
- Use dropout on non-recurrent connections first.
- Clip gradients and inspect validation backtests by horizon.

- **Peephole LSTM:** gates also observe the cell value.
- **Projection LSTM:** reduce hidden dimensionality for speed.
- **ConvLSTM:** replace matrix products with convolutions for grids.
- **Bidirectional LSTM:** useful for classification or imputation, not causal forecasting.

Variants are covered in the RNN source deck.

Regularizing recurrent models

- Standard dropout between stacked recurrent layers.
- Same dropout mask across time for recurrent dropout.
- Weight noise or zoneout when overfitting is severe.
- Early stopping using time-ordered validation windows.
- Avoid evaluating on windows that overlap too closely with training.

Limitations of LSTMs

- Sequential computation is hard to parallelize across time.
- Long-context memory is compressed into a fixed-size state.
- Very long histories still require careful training.
- Cross-channel interactions can be awkward in high-dimensional series.
- Transformers and SSMs often train faster on modern hardware.

Gated recurrent units

Why GRU?

The GRU is a simpler gated recurrent cell.

Compared with LSTM:

- no separate cell state,
- fewer gates,
- often faster and easier to tune,
- usually comparable on medium-sized sequence tasks.

Chung et al., 2014; RNN source deck.

GRU equations

$$\begin{aligned}z_t &= \sigma(W_z x_t + U_z h_{t-1} + b_z), \\r_t &= \sigma(W_r x_t + U_r h_{t-1} + b_r), \\\tilde{h}_t &= \tanh(W_h x_t + U_h (r_t \odot h_{t-1}) + b_h), \\h_t &= (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t.\end{aligned}$$

z_t is the update gate and r_t is the reset gate.

$$\tilde{\mathbf{h}}_t = \tanh(\mathbf{W}_h \mathbf{x}_t + \mathbf{U}_h (\mathbf{r}_t \odot \mathbf{h}_{t-1}) + \mathbf{b}_h).$$

The reset gate decides how much past state is used to form the candidate update.

For time series, this can help after abrupt transitions or changes in operating mode.

Update gate

$$\mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t.$$

The update gate interpolates between keeping old state and writing a new state.

This is the GRU version of controlled memory.

Cell	Strength	Cost
LSTM	explicit memory cell	more parameters
GRU	simpler gating	often faster
Vanilla RNN	minimal state	unstable on long dependencies

Empirically, the best choice is dataset-dependent; start simple and validate by horizon.

Modern recurrent and state-space models

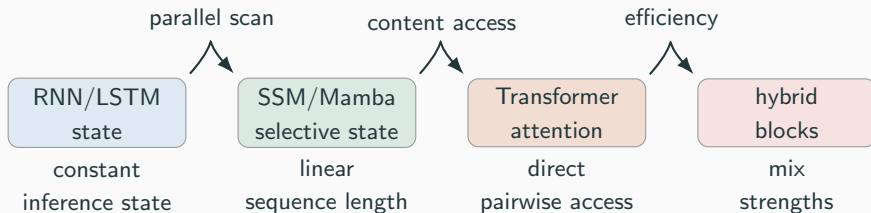
Why recurrence returned

Transformers made sequence modeling highly parallel, but long time series expose costs:

- attention memory grows as $O(T^2)$,
- many industrial streams are long and dense,
- forecasting often needs fast rolling inference,
- stateful models naturally support online updates.

Modern recurrent models try to combine recurrent inference with parallel training.

A modern backbone map



The current trend is not simply “RNN vs Transformer”, but choosing the right memory primitive.

Many modern sequence layers can be written as:

$$\mathbf{s}_t = \mathbf{A}_t \mathbf{s}_{t-1} + \mathbf{B}_t \mathbf{x}_t, \quad \mathbf{y}_t = \mathbf{C}_t \mathbf{s}_t.$$

If the update is associative, training can use parallel scan algorithms.

This gives recurrent-style inference and GPU-friendly training.

RWKV is a recurrent architecture designed to look Transformer-like during training.

- It uses a form of linear attention.
- It can be expressed as a Transformer-style computation or an RNN-style state update.
- Inference has constant memory with respect to context length.

Peng et al., "RWKV: Reinventing RNNs for the Transformer Era", 2023.

xLSTM revisits LSTM scaling with modern LLM-style blocks.

- **sLSTM**: scalar memory with exponential gating.
- **mLSTM**: matrix memory and parallelizable covariance-like updates.
- Residual stacking turns cells into deep sequence backbones.

Beck et al., "xLSTM: Extended Long Short-Term Memory", 2024.

xLSTMTIME adapts xLSTM to long-term multivariate forecasting.

- Motivation: recurrent memory can model temporal dynamics efficiently.
- xLSTM adds stronger gating and memory capacity than classical LSTM.
- The paper reports competitive results against recent long-term forecasting models.

Alharthi and Mahmood, "xLSTMTIME", 2024.

Classical state-space models describe hidden dynamics:

$$\mathbf{s}_t = \mathbf{A}\mathbf{s}_{t-1} + \mathbf{B}\mathbf{x}_t, \quad \mathbf{y}_t = \mathbf{C}\mathbf{s}_t + \mathbf{d}.$$

Neural SSMs replace fixed hand-designed dynamics with learned high-dimensional filters.

This is a natural fit for signals, sensors, audio, and forecasting.

Structured SSMs made long sequence modeling efficient.

Mamba adds **selectivity**: SSM parameters depend on the current input.

$$\mathbf{A}_t, \mathbf{B}_t, \mathbf{C}_t = \text{functions}(\mathbf{x}_t)$$

This helps the model decide what to propagate or forget based on the current observation.

Gu and Dao, "Mamba: Linear-Time Sequence Modeling with Selective State Spaces", 2023/2024.

Why selectivity matters for time series:

- ignore noisy measurements,
- preserve regime information,
- react to shocks,
- support long contexts with linear scaling.

Mamba is recurrent at inference time, but uses hardware-aware parallel algorithms during training.

A Mamba block usually combines:

- input projection,
- local convolution for short-term mixing,
- selective state-space scan,
- gating and output projection,
- residual connection and normalization.

For time series, this combines local smoothing with long-range state.

Mamba4Cast is a time-series foundation model based on Mamba.

- It targets zero-shot forecasting.
- It generates a whole horizon in one pass.
- It is designed to scale better with prediction length than autoregressive Transformers.

Bhethanabhotla et al., "Mamba4Cast", 2024.

Mamba-2 and state-space duality

Mamba-2 builds on a connection between SSMs and attention-like structured matrices.

- Theoretical view: attention and SSMs are more related than they first appear.
- Practical result: a refined selective SSM core.
- Reported speedups over Mamba while remaining competitive with Transformers.

Dao and Gu, "Transformers are SSMs", ICML 2024.

Hybrid Mamba-Transformer models

Hybrid models mix blocks because different mechanisms solve different problems.

- Mamba/SSM blocks: efficient long-range state.
- Attention blocks: direct retrieval from the visible context.
- MoE blocks: increase capacity without activating all parameters.

For time series, hybrids are attractive when both local dynamics and rare long-range events matter.

Complexity tradeoff

Backbone	Training	Inference state	Long context
RNN/LSTM	sequential	$O(d)$	hard to train
Transformer	parallel	$O(Td)$ cache	expensive attention
SSM/Mamba	parallel scan	$O(d)$	linear in T
Hybrid	mixed	mixed	application-dependent

There is no free lunch; the right memory depends on the task.

Time-series channels often have no natural sequence order.

Example: temperature, pressure, humidity, and vibration are simultaneous variables, not a naturally ordered sequence.

When applying sequence models across channels, be careful:

- temporal order is real,
- channel order may be arbitrary,
- graph or attention mechanisms may be better for channel relations.

Choosing a modern recurrent model

- Use LSTM/GRU for smaller supervised forecasting or classification baselines.
- Use xLSTM-style models when gated memory is central and scale matters.
- Use SSM/Mamba-style models for long dense sequences and fast inference.
- Use hybrids when rare events require direct context access.

- Many modern results are young and benchmark-sensitive.
- Forecasting benchmarks can be fragile under preprocessing choices.
- Simple linear models remain surprisingly strong in long-term forecasting.
- Always compare against seasonal naive and strong classical baselines.

Transformers for time series

Why attention?

An RNN compresses the past into a fixed-size state.

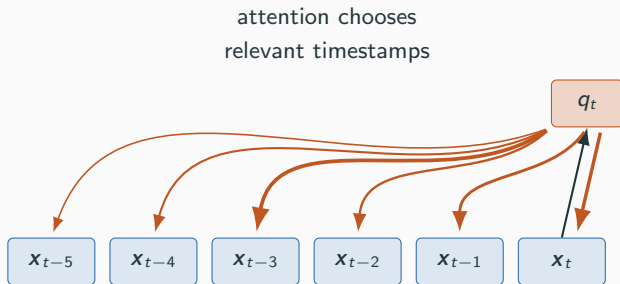
Attention lets a model directly look back at visible timestamps:

$$\hat{\mathbf{y}}_t = f(\mathbf{x}_t, \mathbf{x}_{t-1}, \dots, \mathbf{x}_{t-L+1}).$$

This is useful when a forecast depends on a specific old event, not just a smooth state.

Transformer mechanics follow the Kaiser Transformer source deck, reframed for time series.

Self-attention over time



For each timestamp, attention computes a weighted average of information from other timestamps.

For each time step:

$$\mathbf{q}_t = \mathbf{W}_Q \mathbf{x}_t, \quad \mathbf{k}_t = \mathbf{W}_K \mathbf{x}_t, \quad \mathbf{v}_t = \mathbf{W}_V \mathbf{x}_t.$$

- Query: what information does this step need?
- Key: what information does this step offer?
- Value: what content is passed forward?

Scaled dot-product attention

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right) \mathbf{V}.$$

The matrix $\mathbf{Q}\mathbf{K}^\top$ scores all pairs of timestamps.

For causal forecasting, future timestamps are masked.

One attention pattern is not enough.

Different heads can specialize:

- recent trend,
- daily seasonality,
- weekly seasonality,
- event-driven jumps,
- cross-channel relations.

Attention is permutation-invariant without position information.

Time-series encodings may include:

- absolute or relative position,
- time-of-day, day-of-week, month,
- age of observation,
- irregular time gaps,
- known future calendar features.

Many time-series Transformers use patches:

$$[x_1, \dots, x_P], [x_{P+1}, \dots, x_{2P}], \dots$$

Why patch?

- reduce effective sequence length,
- encode local shape,
- make attention cheaper,
- align with multi-step forecasting heads.

Encoder vs decoder forecasting

Architecture	Typical use	Masking
Encoder-only	representation, imputation, classification	bidirectional
Decoder-only	causal forecasting	causal
Encoder-decoder	history-to-horizon mapping	mixed

The correct choice depends on what is known at prediction time.

A decoder-only time-series model predicts future patches autoregressively:

$$p(\mathbf{y}_{t+1:t+H} \mid \mathbf{x}_{1:t}) = \prod_{k=1}^H p(y_{t+k} \mid \mathbf{x}_{1:t}, y_{t+1:t+k-1}).$$

TimesFM follows this patched decoder-only foundation-model direction.

Encoder-only representation

Encoder-only models are useful when the whole window is visible.

- classify an ECG segment,
- detect anomalies in an already observed window,
- impute missing values,
- pretrain representations by masking patches.

They are not causal unless carefully masked.

- RNN: information from $t - L$ reaches t after L transitions.
- Dilated convolution: path can be logarithmic in window length.
- Self-attention: any visible timestamp can interact in one layer.

This direct path is a major reason attention became dominant.

Full attention over T timestamps forms a $T \times T$ score matrix.

memory and compute $\sim O(T^2)$.

For long time series this is often the bottleneck.

Patching, sparse attention, linear attention, and SSMs all attack this cost.

Forecasting models often output a distribution:

$$\mu_{t+h}, \sigma_{t+h} = \text{Head}_h(\mathbf{z}_t),$$

$$y_{t+h} \sim \mathcal{N}(\mu_{t+h}, \sigma_{t+h}^2).$$

Other choices include quantile heads, mixture distributions, or discrete token sampling.

Time-series Transformers can use known future covariates:

- calendar features,
- planned prices or promotions,
- weather forecasts,
- control inputs,
- holiday and event indicators.

The model must separate known future inputs from unknown future targets.

Long-horizon pitfalls

- Autoregressive decoding accumulates errors.
- Direct heads can blur multi-modal futures.
- Attention may overfit repeated calendar artifacts.
- Good average error can hide poor tail-risk behavior.
- Forecasts should be inspected by horizon, not only averaged.

When Transformers win

Transformers are strong when:

- the dataset is large enough,
- direct access to old timestamps matters,
- multivariate interactions are complex,
- pretraining across many datasets is possible,
- hardware favors parallel matrix multiplications.

What after Transformers?

Recent work asks whether one pretrained model can forecast many domains.

Motivation:

- avoid training one model per dataset,
- support zero-shot or few-shot forecasting,
- reuse patterns across industries and granularities,
- simplify forecasting pipelines.

Chronos tokenizes real-valued time series using scaling and quantization.

- Uses Transformer language-model architectures on time-series tokens.
- Trains with cross-entropy.
- Produces probabilistic forecasts by sampling token sequences.
- Reports strong zero-shot performance across many datasets.

Ansari et al., "Chronos: Learning the Language of Time Series", 2024.

TimesFM is a decoder-only foundation model for forecasting.

- Uses patched time-series inputs.
- Handles different history lengths, horizons, and granularities.
- Targets zero-shot forecasting on public datasets.
- Shows how Transformer scaling ideas transfer to numeric sequences.

Das et al., “A decoder-only foundation model for time-series forecasting”, 2024.

Moirai:

- universal forecasting Transformer,
- handles cross-frequency and arbitrary variate counts,
- trained on LOTSA, a large open time-series archive.

MOMENT:

- open time-series foundation-model family,
- uses the Time-series Pile,
- supports multiple tasks with limited supervision.

Generative time-series models can represent many possible futures.

- Diffusion-style models iteratively denoise future trajectories.
- They are attractive when futures are multi-modal.
- They can be slower than direct forecasting heads.
- For decision making, sample quality and calibration matter.

This direction is still less settled than Transformer and SSM forecasting.

Memory, retrieval, and filtering

Post-Transformer ideas often reintroduce explicit memory.

- Retrieval: find similar historical windows or events.
- Test-time memory: update a memory module during inference.
- Kalman-style linear attention: combine Bayesian filtering ideas with parallel scans.
- Hybrid models: use attention for short-term lookup and state for long-term memory.

See Titans-style neural memory and Kalman Linear Attention for recent examples.

Summary

Model selection checklist

- Short sequence, small data: start with GRU/LSTM and strong classical baselines.
- Long dense stream: consider SSM/Mamba-style backbones.
- Need direct access to past events: consider attention or hybrids.
- Need zero-shot forecasting: evaluate time-series foundation models.
- Need uncertainty: choose probabilistic heads and calibrate forecasts.

Sources and further reading

- Jan Chorowski, *Recurrent Neural Networks*, TMLSS 2018 slides: 10-11-RNNs-from.TMLSS.pdf.
- Lukasz Kaiser, *Tensor2Tensor Transformers*, 2018 slides: 12_transformer_from_lkaiser.pdf.
- Hochreiter and Schmidhuber, *Long Short-Term Memory*, 1997.
- Chung et al., *Empirical Evaluation of Gated Recurrent Neural Networks*, 2014.
- Beck et al., *xLSTM: Extended Long Short-Term Memory*, 2024.
- Gu and Dao, *Mamba: Linear-Time Sequence Modeling with Selective State Spaces*, 2023/2024.
- Dao and Gu, *Transformers are SSMs*, ICML 2024.
- Alharthi and Mahmood, *xLSTMTime*, 2024.
- Bhethanabhotla et al., *Mamba4Cast*, 2024.
- Ansari et al., *Chronos*, 2024; Das et al., *TimesFM*, 2024; Woo et al., *Moirai*, 2024; Goswami et al., *MOMENT*, 2024.